YTPS5 · PLAINENGLISHGUIDE

How ytps5 blocks YouTube ads on a PS5

No programming knowledge required.

What the app really is · how an ad normally happens · how a hook works · why settings look native · what an SDK is · why firmware matters

The 60-second story	A cinema, a printed list, and a very fast assistant.
The words on the box	Every technical term, in plain language.
How an ad really arrives	Why an ad is a line item, not a video file.
The eight steps	From loading the payload to a clean playback.
Toast and settings	How ytps5 borrows the app's own menus.
SDK and firmware	Why an update moved the furniture.
Risks and FAQ	The honest limits, and the questions people ask.

Version 1.1 · November 2026

Every explanation here comes from the project's public source code.

What's inside

This guide answers three questions: **what** the PS5 YouTube app is, **how** ads get into it, and **what ytps5 does** to stop them. Every technical word is explained the first time it appears.

1. The 60-second story 3

2. The words on the box 3

3. What the PS5 YouTube app really is 5

4. How an ad normally happens 5

5. What ytps5 does, step by step 6

6. The ad filter: crossing names off the guest list 7

7. SponsorBlock: skipping what is inside the video 8

8. The toast and the settings menu 9

9. What an SDK is, and why firmware broke it 10

10. Why nothing on disk changes 11

11. What it does not do 11

12. Risks and honest limits 11

13. How it was built and tested 12

14. Questions people actually ask 12

15. The whole thing in six lines 13

> **NOTE**
> Nothing in this guide is a guess. The explanations come from the project's own source code, which is public. Where something is still unknown or unverified, the text says so.

1. The 60-second story

Imagine a cinema. The building, the screen and the seats are the YouTube app. The film is the video you picked.

Before each film, the cinema receives a printed list from head office. The list says: play this trailer first, then these three ads, then the film. The projectionist follows the list.

ytps5 is a tiny assistant standing next to the projectionist. Every time a list arrives, the assistant crosses out the lines that mention ads, **before** the projectionist reads it. The cinema never learns that ads were planned. It simply plays the film.

Nothing in the building is rebuilt or replaced. The list is edited in memory, every single time. Close the app, and the next list arrives clean because the assistant is no longer there — which is why you start ytps5 again after a reboot.

For sponsor reads that are baked into the film itself — the host saying "this video is brought to you by..." — there is nothing to cross out. For those, the assistant keeps a community-made list of timestamps and fast-forwards past them.

> **IN ONE SENTENCE**
> ***ytps5 does not modify the YouTube app. It changes the data the app reads while it is running.***

2. The words on the box

Skip this page now if you like and come back when a word confuses you. It is a dictionary, not a chapter.

Word	What it actually means
Firmware	The console's built-in operating software. Like the version of iOS on a phone. 8.20, 11.40 and 13.60 are firmware versions.
Jailbreak	Unlocking the console so it will run programs that Sony did not sign. Without it, nothing in this guide can run.
Payload	A small program you load after jailbreaking. ytps5 is a payload.
ELF	The file format of a payload on PlayStation. Think of it as the .exe of the PlayStation world. `ytps5.elf` is one ELF file.
Process	A program while it is running. The YouTube app is one process; ytps5 is another.
Memory (RAM)	The desk where a running program keeps its papers. It is wiped when the program closes.
Kernel	The most privileged part of the console's operating system. It owns the list of running programs and their memory.

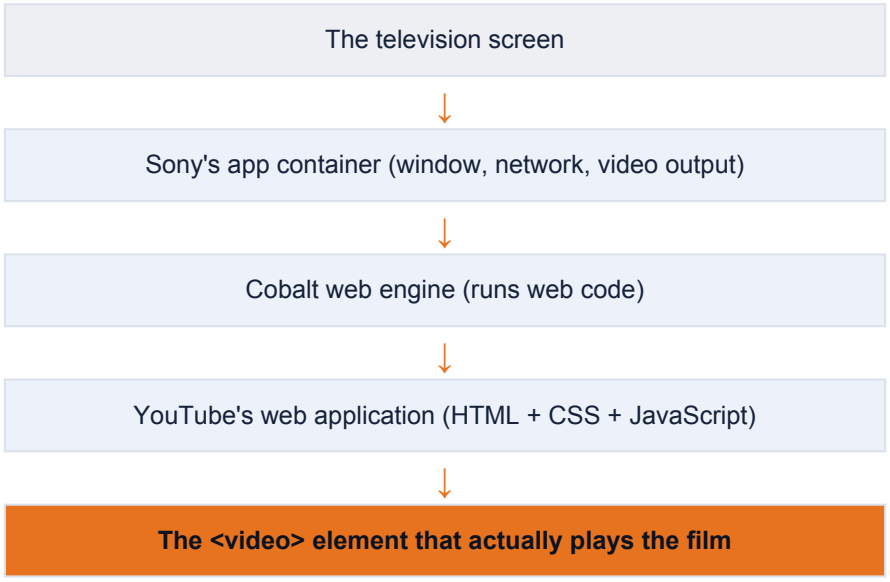
Word	What it actually means
SDK	Software Development Kit. A toolbox that lets a computer build programs for a particular machine: compiler, headers, libraries, linker. Explained properly in section 9.
Compiler	A translator. It turns human-readable code into the machine instructions the console understands.
Library	A box of pre-built code you can reuse instead of writing everything yourself.
Hook	A detour sign. We redirect one function to our own code and then send it back on its way.
JavaScript	The programming language the YouTube app's interface is written in.
JSON	A text format for structured data. YouTube sends the app things like "here is the video URL, the title, the quality options, and two ad slots" as JSON.
API	A menu of requests a service accepts. The SponsorBlock API accepts "give me the skipped segments for this video".
Proxy	A middleman. Our program asks the proxy, and the proxy asks the internet.
SponsorBlock	A free, public database of timestamps for sponsor reads and similar segments, filled in by viewers.
Toast	The small popup that appears at the bottom of the screen for a few seconds.
Renderer	The part of the app that draws a piece of the screen from a description. The app has renderers for menus, buttons and toasts.
Address / offset	A number that says where something lives in memory, like an apartment number in a very large building.
Title ID	The short code that identifies an app. The YouTube app tested here is `PPSA01652`.

> **NOTE**
> If you remember only four words from this page, remember these: kernel (owns the process list), SDK (the toolbox and the address book), hook (a detour in a function), and JSON (the data packages the app reads).

3. What the PS5 YouTube app really is

This is the part most people get wrong. The YouTube app on a PS5 is not a normal game. It is a **web page inside a Sony box**.

Sony ships a container that knows how to talk to the TV, the network and the video hardware. Inside that container runs a web engine called Cobalt, which is the same family of technology as a browser. Inside Cobalt runs YouTube's own web application, written in HTML, CSS and JavaScript.



Four layers. The top two are ordinary web code, which is why this project is possible at all.

Because the interface is web code, a program that can reach into the app's memory can change what that web code does. It does not need to rebuild the app, patch the installed package, or touch Sony's signature. It edits the running copy.

A native game has no such layer. That is why this technique works for the YouTube app and not for a random game.

> **IN ONE SENTENCE**
>
> ***The YouTube app is a browser tab that Sony dressed up as a console app.***

4. How an ad normally happens

An ad on YouTube is not a separate program and not a separate app. It is a line item in a data package.

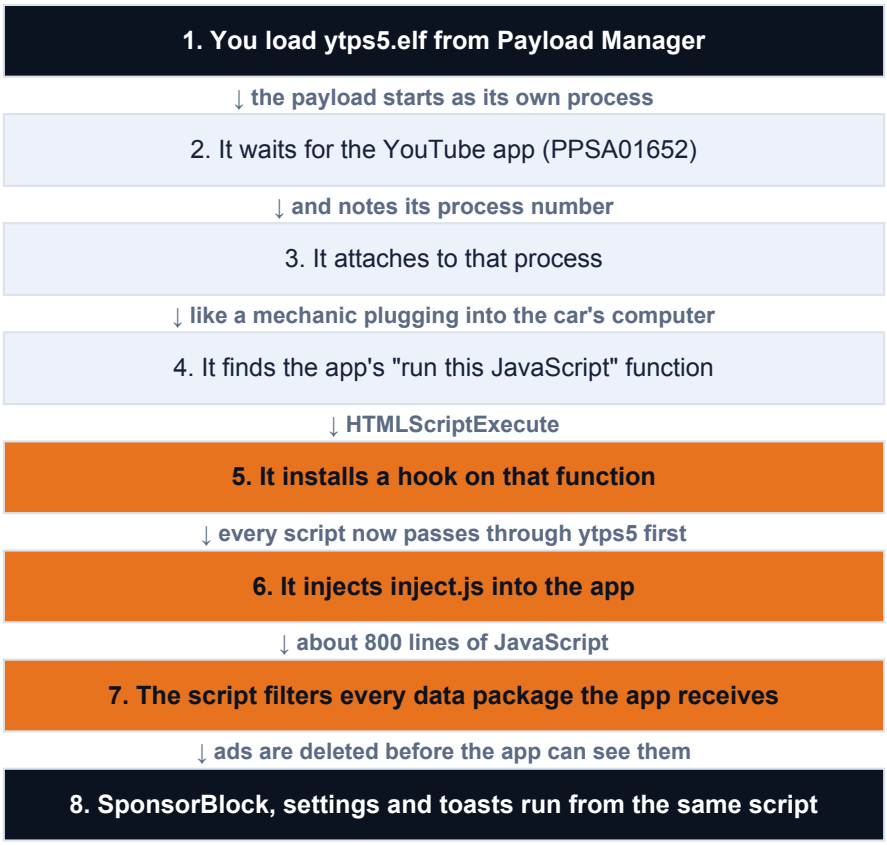
1. You highlight a video and press play.
2. The app asks YouTube's servers: **"give me the plan for this video."**
3. The server replies with JSON: the video address, the available qualities, the title, the buttons, the recommended videos — **and the ad slots**.
4. The app reads the package. If the package says there is an ad, the app plays the ad first.

Two consequences follow from this, and they explain the whole design of ytps5:

- **The ad can be removed by removing the line item.** If the app never sees an ad slot in the package, it never asks for an ad.
- **Blocking YouTube's ad servers is a worse idea.** The app expects a reply. A blocked request can leave the player waiting, retrying, or showing an error. Editing the package is cleaner and invisible.

5. What ytps5 does, step by step

Here is the whole machine, from the moment you run it to the moment a video plays without ads.



Step 1 — Run the payload

After the console is jailbroken, a tool such as Payload Manager can load an ELF file. ytps5 is that file. It runs as its own process, separate from YouTube.

Step 2 — Find the app

The payload watches the list of running programs until the YouTube app appears. It identifies it by its title ID, `PPSA01652`. It does not guess by name; the code is exact.

Step 3 — Attach

Attaching means asking the kernel for permission to inspect another process's memory. It is the same mechanism a debugger uses. From this point ytps5 can read and write the app's memory, but only while the app is running.

> **ANALOGY · the mechanic**
> A mechanic does not rebuild your engine to read its error codes. They plug a diagnostic tool into the port the car already has. Attaching is that port.

Step 4 — Find the doorway

The app runs JavaScript through a single function, `HTMLScriptExecute`. Everything the app draws — the home screen, the player, the settings menu — is produced by scripts that pass through this doorway. Find the doorway, and you can see every script.

Step 5 — Install the hook

A hook is a detour. The first few bytes of `HTMLScriptExecute` are replaced with a jump to a small piece of our code. That code does its work, then jumps back to the original function so the app never notices.

> **ANALOGY · the detour sign**
> A road crew puts a sign in front of a junction: "all traffic goes this way for a moment". Every car still reaches its destination; it just passes the crew's checkpoint first.

Step 6 — Inject the script

ytps5 carries a JavaScript file inside itself. When a page script comes through the doorway, ytps5 makes a copy with its own script glued to the front, and hands that to the app instead. The app runs ytps5 first, then its own code. A guard at the top prevents it from running twice.

Step 7 — Filter the data

The JavaScript replaces the app's `JSON.parse` function — the step where a data package is unpacked into usable objects. The replacement calls the real parser, then walks the result and deletes the ad entries. Section 6 lists exactly what it deletes.

Step 8 — Everything else

The same script also handles SponsorBlock, the settings panel, the in-app toast and the update check. Sections 7 and 8 explain those.

6. The ad filter: crossing names off the guest list

When the app receives a data package, it calls `JSON.parse` to turn the text into objects. ytps5 wraps that function:

1. The app calls `JSON.parse` with a package.
2. Our version calls the **original** `JSON.parse` first, so nothing is lost.
3. It then walks the result and deletes keys whose names mean "ad": `adPlacements`, `playerAds`, `adSlots`, `adBreakHeartbeatParams`, and promoted feed cards.
4. It returns the cleaned object to the app.

The app then behaves as if the package never contained ads. There is no ad to play, so it plays the video.

Feed ads — the sponsored card between recommended videos — are removed the same way, by looking for keys, labels and badges that mark a card as promoted.

> **ANALOGY · the guest list**
> A bouncer does not fight the guests. The bouncer reads the list and crosses off the names that should not be let in. By the time the door opens, the unwanted guest was never invited.

> **WORTH KNOWING**
> This is a filter, not a wall. If YouTube invents a new name for an ad field tomorrow, the filter has to learn it. That is why the project is open source and why updates matter.

7. SponsorBlock: skipping what is inside the video

Some ads are not separate clips. They are part of the video: the host talking about a VPN, an intro animation, a recap. You cannot delete them from a data package because they are already inside the film.

SponsorBlock is a public database, built by viewers, that records the start and end time of those segments. ytps5 uses it like this:

1. When a video starts, the script asks for the segments for that video ID.
2. The request goes to a small local proxy at `127.0.0.1:4040` on the console. The proxy forwards it to `sponsor.ajay.app`, adds the categories, and caches the answer.
3. The script watches the video's playback position.
4. When playback is about to enter a recorded segment, it jumps the position to the end of the segment.
5. A toast appears: "Segment Skipped".

The proxy exists for two reasons: the app's web sandbox is fussy about which servers it may contact, and the proxy can cache answers so the same video does not hit the API twice.

> **ANALOGY · the bookmark**
> You are reading a book with a friend who has already read it. They put a sticky note on the boring pages. You do not tear the pages out; you just skip to the next sticky note.

8. The toast and the settings menu

This is the part that surprises people most: ytps5 does not draw its own pixels. It asks the YouTube app to draw things using the app's own components. That is why the result looks native.

Two different toasts

- **The console notification** (top-left, from the payload) reports progress: "attaching", "hook installed", "1 script injected". This is the PS5 system notification, not part of YouTube.
- **The in-app toast** (bottom of the video) is produced by asking the app to show one of its own toasts. Our script sends the app a command of type `openPopupAction` with `overlayToastRenderer`. The app renders it exactly as it renders its own "Added to Watch Later" messages.

That is the whole trick: we speak the app's language instead of painting on top of it.

Replacing the Premium row

The settings menu is also data. YouTube sends a list of rows; each row has a label and an action. Our filter walks that list and does three things:

- removes rows labelled **Purchases** or **Memberships**;
- finds the row labelled **Premium** and replaces it with a row labelled **ytps5 Settings**;
- gives that row a private action name, `YTPS5_SETTINGS_SHOW`, which the real app does not know.

Our script also wraps the app's command handler, `resolveCommand`. When a command with our private action arrives, we intercept it and open our own panel instead of passing it on.

The panel itself is built from the app's own building blocks — `settingsModal`, `settingsItemList`, `settingsButton` — so it inherits the typography, spacing, focus behaviour and controller navigation of the real settings menu. It contains Ad Block, SponsorBlock, the category toggles, a support link and an update check.

> **ANALOGY · the menu swap**
> A restaurant prints its menu on a standard template. You swap one page for a page of your own that uses the same template, same font and same paper. The waiter cannot tell the difference, and the kitchen never reprinted anything.

> **WORTH KNOWING**
> The Premium row is not deleted from the app's code. It is replaced in the data every time the menu is built. Start the app without ytps5 and the original row is back. Nothing on disk changed.

9. What an SDK is, and why firmware broke it

An SDK — Software Development Kit — is the toolbox that lets a computer build programs for a specific machine. For a PS5 it contains four things:

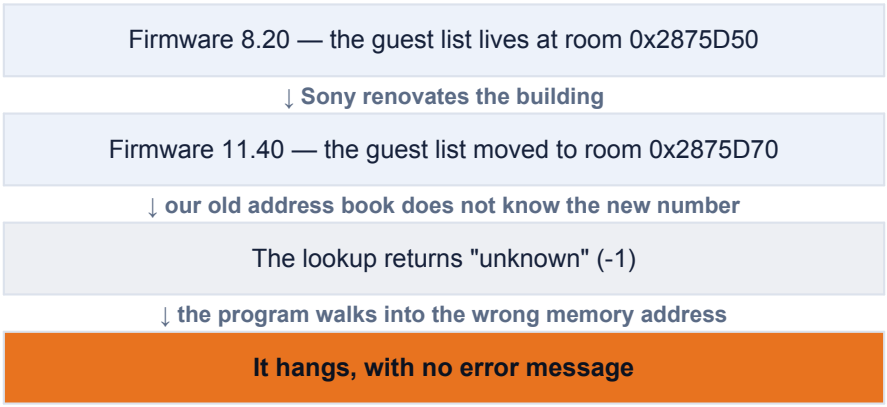
- **A compiler**: the translator from human code to PS5 machine code.
- **Headers**: the dictionaries that tell the compiler what the console's functions are called and what they expect.
- **Libraries**: pre-built code for common jobs, so you do not rewrite them.
- **Linker and startup files**: the instructions for turning the pieces into one loadable file.

Sony does not publish this toolbox for hobbyists. The community built one, called the PS5 payload SDK. ytps5 is compiled with it on an ordinary Mac, and the result is a single ELF file.

Why a firmware update can break a payload

The kernel keeps the list of running programs in memory. To read that list, a program needs its address — the apartment number of the building where the list lives.

Every firmware update is a renovation. Walls move, rooms are renumbered. The address that worked on 8.20 points somewhere else on 11.40. A program carrying an old address book knocks on the wrong door.



That is exactly what the 11.40 log showed: every address came back as `0xffffffffffffffff`, and the log stopped at "attaching".

The fix, and why it generalises

The SDK is the up-to-date directory at the front desk. It already knows the new room numbers, because its authors update it for each firmware. Our code was carrying its own private address book, copied from an older project that stopped at firmware 10.60.

For 11.40 we copied the correct numbers from the SDK's own table and from a public project called kstuff-lite. The better long-term fix is to stop carrying a private address book at all and ask the SDK every time. Then the project supports every firmware the SDK supports — currently up to 13.60 in the copy bundled here, and 14.22 in the newest upstream SDK.

> **IN ONE SENTENCE**
>
> ***Supporting more firmware is mostly a data problem, not a hardware problem — but only a real console can confirm the data is right.***

10. Why nothing on disk changes

It is worth being precise about what is and is not touched.

	The installed app	The running app
Where it lives	The console's storage	Memory (RAM)
What ytps5 does	Nothing	Reads and edits the running copy
After you close the app	Unchanged	Wiped by the system
After a reboot	Unchanged	Needs ytps5 again

There is no modified app package, no patched installation, no replacement of Sony's files, and no change to your account. The payload lives in its own folder and runs as its own program.

This is also why the project can distribute a single ELF file and nothing else. It ships no app, no game, no key and no decryption tool.

11. What it does not do

- It does not modify, repack or redistribute the YouTube app.
- It does not contain Sony keys, decryption tools, DRM bypasses or HDCP bypasses.
- It does not unlock paid content, movies, games or PlayStation Plus.
- It does not touch game backups, save data or your account.
- It does not block ads in other apps, on other devices, or in a browser.
- It does not guarantee every ad. It removes the ad entries it knows about; new formats need new rules.

The project is a runtime research tool. It is not a piracy tool, and the repository deliberately contains nothing that would make it one.

12. Risks and honest limits

- **Jailbreaking has its own risks**, independent of this project: instability, lost access, and the chance of a bricked console if you use a bad exploit.
- **Firmware updates undo jailbreaks.** Do not update if you want to keep using payloads.
- **Unknown firmware can hang or crash.** A wrong address that is only read can stall; one that is written can panic the console. Test builds for untested firmware are labelled experimental for that reason.

- **YouTube can change the app at any time.** New field names, new player logic or a new app build can break the filter until it is updated.
- **Terms of service.** Using a modified runtime may violate YouTube's terms. Whether an account is affected is unknown, and no one can promise otherwise.
- **No affiliation.** The project is not endorsed by Sony or YouTube.

13. How it was built and tested

There is no Sony development kit and no leaked code involved. The process is ordinary software engineering with unusual constraints:

1. Compile the payload on a Mac with the community SDK.
2. Copy the resulting `ytps5.elf` to the console.
3. Load it with Payload Manager on a jailbroken console.
4. Watch the log file and the console notification to see how far it got.
5. Open the app, play a video, and check the result on screen.
6. Change one thing, rebuild, repeat.

The first working target was firmware 8.20 with YouTube `01.000.003` (`PPSA01652`). Screenshots and logs from that console are the evidence behind every claim in this guide.

When a user on firmware 11.40 sent his log folder, the failure was visible in one line: every kernel address was unknown. That is how the fix in section 9 was found without owning an 11.40 console — and why the next step is a small test build for volunteers rather than a purchase.

14. Questions people actually ask

Do I need a computer every time?

No. You need one to download the file. After that, the payload is loaded from the console itself.

Do I have to start it after every reboot?

Yes. The patch lives in memory, and memory is cleared at shutdown. Depending on your jailbreak, you may also have to redo the jailbreak.

Why does the settings row say ytps5?

Because the Premium row in the settings data was replaced with our row while the app is running. Section 8 explains it.

Does it work on firmware 11.40?

A test build exists and is waiting for a report from the volunteer who reported the issue. Until that log arrives, 11.40 is untested.

Will it work on 13.60 or 14.x?

The code can be made to work wherever the SDK knows the addresses. That is a code change, not a hardware purchase. Verification still needs one console per firmware.

Why not just block the ad domains?

The app expects an answer. Blocking can stall the player or trigger retries. Removing the ad entries from the data is cleaner.

Does it block every ad?

It blocks the in-player and feed ads it knows about. New formats need new rules. No client-side tool can promise 100% forever.

Is it free?

Yes. The project is open source under GPL-3.0. Sponsorship is optional.

15. The whole thing in six lines

1. The PS5 YouTube app is a web page in a Sony box.
2. Ads are entries in the data YouTube sends before a video.
3. ytps5 attaches to the running app and hooks its "run JavaScript" function.
4. It injects a script that deletes ad entries before the app reads them.
5. SponsorBlock timestamps handle ads that are inside the video.
6. Nothing on disk changes; closing the app restores everything.

> **WHERE TO FIND THINGS**
> Repository: github.com/MasterShivnath007/ytps5-sponsorblock
> Downloads: github.com/MasterShivnath007/ytps5-sponsorblock/releases
> SponsorBlock: sponsor.ajay.app